

Python Tuple Objects

A *tuple* in Python is just an ordered collection of some Python objects.

To create a tuple, write some Python objects seperating them with a: `,`

In [18]:
```
'hello', 'world', 1, None, 1
```

Out[18]:
```
('hello', 'world', 1, None, 1)
```

Warning: When representing a tuple object, like in the previous example, Python always adds a pair of paranthesis around the tuple.

Writing the tuples in this way is a good practice.

Python also allows one more `,` at the end of the tuple. This extra `,` does not have an effect. The tuple created will be the same.

In [19]:
```
'hello', 'world', 1, None, 1,
```

Out[19]:
```
('hello', 'world', 1, None, 1)
```

So if you want to create a tuple with single object then just write the object and put a `,` at the end.

In [20]:
```
'hello',
```

Out[20]:
```
('hello',)
```

Warning: Writing `('hello')` without `,` does not create a tuple!

If you want to create an empy tuple use:

In [21]:
```
()
```

Out[21]:
```
()
```

or

In [22]:
```
tuple()
```

Out[22]:
```
()
```

Since a tuple is one Python object then you can assign it to a variable.

In [23]:
```
h = 'hello',
w = 'world', 1
```

Converting Strings To Tuples

In [24]:
```
tuple('hello')
```

```
Out[24]: ('h', 'e', 'l', 'l', 'o')
```

```
In [25]: tuple(('hello',))
```

```
Out[25]: ('hello',)
```

Then you can use `.join()` method to convert a tuple to a string.

```
In [26]: "".join(('h', 'e', 'l', 'l', 'o'))
```

```
Out[26]: 'hello'
```

```
In [27]: ":".join('hello')
```

```
Out[27]: 'h:e:l:l:o'
```

Operations on Tuples

Like strings:

You can get the length of a tuple by the builtin `len()` function.

```
In [28]: len(('Hello', 'World!'))
```

```
Out[28]: 2
```

To test if an item is in the tuple use `in` keyword.

```
In [29]: 'World' in ('Hello', 'World!'), 'Hello' in ('Hello', 'World!')
```

```
Out[29]: (False, True)
```

To test if an item is **not** in the tuple use `not in` keyword.

```
In [30]: 'World' not in ('Hello', 'World!')
```

```
Out[30]: True
```

To count the number of occurences of an item in a tuple use `.count()` method.

```
In [31]: ('Hello', 'World!').count('Hello')
```

```
Out[31]: 1
```

Python tuples can be concatenated by `+` operation.

```
In [32]: ('Hello', 'World') + ('!',)
```

```
Out[32]: ('Hello', 'World', '!')
```

Also you can use `*` to repeat the same tuple multiple times.

```
In [33]: ('Hello', 'World!') * 3
```

```
Out[33]: ('Hello', 'World!', 'Hello', 'World!', 'Hello', 'World!')
```

Precedence of the `,` Operator

Warning: Precedence of the `,` operator is less than the other operators except the `=` operator. Therefore, when doing tuple operations, always use paranthesis around the `,` separated tuple items.

For example:

In [34]:
```
1, 2 + 3, 4
```

Out[34]:
```
(1, 5, 4)
```

is equivalent to:

In [35]:
```
1, (2 + 3), 4
```

Out[35]:
```
(1, 5, 4)
```

but not equivalent to:

In [36]:
```
(1, 2) + (3, 4)
```

Out[36]:
```
(1, 2, 3, 4)
```

Nested Tuples

Paranthesis is used to create a tuple with another tuple:

In [37]:
```
len(('one', 2, (3, (4, 5))))
```

Out[37]:
```
3
```

Here the items of the tuple are: `'one'`, `2` and `(3, (4, 5))`.

Accessing Tuple Items

To access the first item in a tuple use `[0]`, to access the second `[1]`, and so on.

In [38]:
```
('one', 2, (3, (4, 5)))[0]
```

Out[38]:
```
'one'
```

To access the last item of a tuple use `[-1]`. You can use negative numbers to index the items from end.

In [39]:
```
('one', 2, (3, (4, 5)))[-1]
```

Out[39]:
```
(3, (4, 5))
```

To index a tuple from the beginning count like `0,1,2,...` To index a tuple from the end count like `-1,-2,-3,...`

To get the (positive) index of an item in a tuple use `.index()` method.

In [40]:
```
('one', 2, (3, (4, 5))).index((3, (4, 5)))
```

Out[40]:
```
2
```

To access an item in a nested tuple use multiple `[...]`'s.

In [41]:
```
('one', 2, (3, (4, 5)))[-1][1]
```

Out[41]:
```
(4, 5)
```

In [42]:
```
('one', 2, (3, (4, 5)))[-1][1][0]
```

Out[42]:
```
4
```

Since we use the same notation to access the letters of a string, be careful when working on tuples of strings.

In [43]:
```
('one', 2, (3, (4, 5)))[0][0]
```

Out[43]:
```
'o'
```

Here `[0][0]` is the first character of the first item in the tuple.

Range of Indexes

You can get a part of a tuple by `[start:end]` notation.

In [44]:
```
('one', 2, (3, (4, 5)))[0:2]
```

Out[44]:
```
('one', 2)
```

Here `[0:2]` gets a tuple with `[0]` and `[1]` but not `[2]`. You can also use negative indexes here.

In [45]:
```
('one', 2, (3, (4, 5)))[0:-1]
```

Out[45]:
```
('one', 2)
```

If you ommit the start index Python assumes it `0`. So to get the first `2` items:

In [46]:
```
('one', 2, (3, (4, 5)))[:2]
```

Out[46]:
```
('one', 2)
```

If you ommit the end index Python assumes it the length of the tuple. So to get the last `2` items:

In [47]:
```
('one', 2, (3, (4, 5)))[-2:]
```

Out[47]:
```
(2, (3, (4, 5)))
```

If your indexes are out of valid ranges then Python assumes them like empty:

In [48]:
```
('one', 2, (3, (4, 5)))[-10:10]
```

Out[48]:
```
('one', 2, (3, (4, 5)))
```

You can skip some items with `[start:end:skip]` notation. For example to get only the items in odd positions:

In [49]:
```
('one', 2, (3, (4, 5)))[::2]
```

Out[49]:
```
('one', (3, (4, 5)))
```